

EXPERIMENT 1:

Aim

To write a Python program that solves the 8-puzzle problem using the A* search algorithm.

Objective

- To represent the 8-puzzle board and simulate moving the blank tile (UP, DOWN, LEFT, RIGHT).
- To implement the A* algorithm using Manhattan distance to find the shortest path to the goal state.
- To print each step showing how the puzzle is solved.

Algorithm

1. Set up the target goal state (1 to 8, with 0 as the blank space).
2. Create an open_list (priority queue) for states to explore and a closed_set to keep track of visited states.
3. For the current board, calculate the **Manhattan Distance** (how far each tile is from its target spot).
4. Calculate total cost: $f(n) = g(n) + h(n)$, where:
 - $g(n)$ = moves made so far.
 - $h(n)$ = Manhattan distance heuristic.
5. Push the starting board into open_list.
6. Loop until open_list is empty:
 - Pick the state with the lowest $f(n)$ value.
 - If it matches the goal state, trace back the steps and stop.
 - Otherwise, mark it as visited.
 - Generate all possible moves by sliding the blank tile (0) up, down, left, or right.
 - Add new, unvisited states to open_list.

Code:

```
*aiexp1.py - C:\Users\Admin\OneDrive\Documents\aiexp1.py (3.14.7)*
File Edit Format Run Options Window Help

1 import heapq
2 GOAL_STATE = (
3     (1, 2, 3),
4     (4, 5, 6),
5     (7, 8, 0)
6 )
7
8 class PuzzleNode:
9     def __init__(self, board, parent=None, move="", g=0):
10         self.board = board
11         self.parent = parent
12         self.move = move
13         self.g = g
14         self.h = self.get_manhattan_distance()
15         self.f = self.g + self.h
16
17     def get_manhattan_distance(self):
18         distance = 0
19
20         targets = {
21             1: (0, 0), 2: (0, 1), 3: (0, 2),
22             4: (1, 0), 5: (1, 1), 6: (1, 2),
23             7: (2, 0), 8: (2, 1), 0: (2, 2)
24         }
25         for r in range(3):
26             for c in range(3):
27                 val = self.board[r][c]
28                 if val != 0:
29                     tr, tc = targets[val]
30                     distance += abs(r - tr) + abs(c - tc)
31         return distance
32
33     def __lt__(self, other):
34         return self.f < other.f
35
36     def find_blank(self):
37         for r in range(3):
38             for c in range(3):
39                 if self.board[r][c] == 0:
40                     return r, c
41
42     def get_neighbors(self):
43         neighbors = []
44         r, c = self.find_blank()
45
46         moves = {
47             "UP": (r - 1, c),
48             "DOWN": (r + 1, c),
49             "LEFT": (r, c - 1),
50             "RIGHT": (r, c + 1)
51         }
52         for move_name, (nr, nc) in moves.items():
53             if 0 <= nr < 3 and 0 <= nc < 3:
54                 new_board = [list(row) for row in self.board]
55                 new_board[r][c], new_board[nr][nc] = new_board[nr][nc], new_board[r][c]
56                 board_tuple = tuple(tuple(row) for row in new_board)
57                 neighbors.append(PuzzleNode(board_tuple, self, move_name, self.g + 1))
58         return neighbors
59
60 def solve(initial_board):
61     start_node = PuzzleNode(initial_board)
62     open_list = []
63     heapq.heappush(open_list, start_node)
64     closed_set = set()
65     while open_list:
66         current = heapq.heappop(open_list)
67
68         if current.board == GOAL_STATE:
69             path = []
70             while current:
71                 path.append(current)
72                 current = current.parent
73             return path[::-1]
74         closed_set.add(current.board)
75         for neighbor in current.get_neighbors():
76             if neighbor.board in closed_set:
77                 continue
78             heapq.heappush(open_list, neighbor)
79     return None
80
81 def print_board(board):
82     for row in board:
83         print(" ".join(str(tile) if tile != 0 else "_" for tile in row))
84     print()
85
86 if __name__ == "__main__":
87     start_board = (
88         (1, 2, 3),
89         (0, 4, 6),
90         (7, 5, 8)
91     )
92     print("Initial State:")
93     print_board(start_board)
94
95     solution = solve(start_board)
96     if solution:
97         print(f"Solved in {len(solution) - 1} moves!\n")
98         for step, node in enumerate(solution):
99             if step > 0:
100                 print(f"Step {step}: Move Blank {node.move}")
101                 print_board(node.board)
102     else:
103         print("No solution exists for this configuration.")

Ln: 15 Col: 33
```

```
*aiexp1.py - C:\Users\Admin\OneDrive\Documents\aiexp1.py (3.14.7)*
File Edit Format Run Options Window Help

104         return path[::-1]
105     return None
106
107 def print_board(board):
108     for row in board:
109         print(" ".join(str(tile) if tile != 0 else "_" for tile in row))
110     print()
111
112 if __name__ == "__main__":
113     start_board = (
114         (1, 2, 3),
115         (0, 4, 6),
116         (7, 5, 8)
117     )
118     print("Initial State:")
119     print_board(start_board)
120
121     solution = solve(start_board)
122     if solution:
123         print(f"Solved in {len(solution) - 1} moves!\n")
124         for step, node in enumerate(solution):
125             if step > 0:
126                 print(f"Step {step}: Move Blank {node.move}")
127                 print_board(node.board)
128     else:
129         print("No solution exists for this configuration.")

Ln: 55 Col: 30
```

OUTPUT:

Initial State:

```
1 2 3
_ 4 6
7 5 8
```

Solved in 3 moves!

```
1 2 3
_ 4 6
7 5 8
```

Step 1: Move Blank RIGHT

```
1 2 3
4 _ 6
7 5 8
```

Step 2: Move Blank DOWN

```
1 2 3
4 5 6
7 _ 8
```

Step 3: Move Blank RIGHT

```
1 2 3
4 5 6
7 8 _
```

|

Result

The 8-puzzle program was executed successfully. Using the A* algorithm, it found the shortest sequence of moves to reach the goal state from the given start state.

Conclusion

The A* search algorithm effectively solves the 8-puzzle problem. By combining actual path cost with a heuristic estimation (Manhattan Distance), it explores fewer unnecessary paths than standard algorithms like BFS or DFS, making it both fast and reliable.